

Laborarbeit: Cloud-native Anwendung

Prüfungsleistung zur Vorlesung Cloud Infrastructures and Cloud Native Applications, Stand 16.07.2026.

1. Ziel

Sie entwerfen, implementieren und betreiben eine eigene cloud-native Anwendung in einer selbst gewählten Domäne. GridFlex aus dem Exercise Track als Vorlage verwendbar; Sie dürfen dieselben Bausteine verwenden, die Domäne muss aber eine andere sein und die Infrastrukturentscheidungen tatsächlich rechtfertigen.

Bewertet wird nicht, wie viele Technologien Sie installiert haben, sondern ob die Anwendung läuft, reproduzierbar deployt und ob Sie erklären können, warum Sie sie so gebaut haben.

2. Rahmen

Punkt	Vorgabe
Gruppengröße	1–4 Studierende
Plattform	DHBWCloud (https://newstack.dhbw.cloud)
Abgabe	Entweder ein - Link zu einem Git-Repository (öffentlich oder Zugriff für Github-User pfisterer) oder - eine ZIP-Datei (max 100MB). Die Abgabe erfolgt in Moodle.
Beschreibung	README.md im Repository gemeinsam mit dem Code
Demonstration	Screencast

Der gesamte Stack läuft auf der DHBWCloud, von den VMs bis zur Anwendung. Externe Cloud-Dienste sind ausgeschlossen: keine gehosteten Datenbanken, kein gehostetes Kubernetes, kein Managed-Grafana, keine externen LLM-APIs. Erlaubt sind Registries und Paketquellen als reine Bezugsquellen sowie Ihr Git-Hoster für den Code.

Es gibt keinen schriftlichen Bericht in PDF-Form. Die **README.md** ersetzt ihn vollständig; ihr Umfang wird über die geforderten Abschnitte gesteuert, nicht über eine Seitenzahl.

3. Anforderungen an die Anwendung

Die Anwendung muss so beschaffen sein, dass die Infrastruktur überhaupt nötig ist:

- Mindestens zwei selbst geschriebene Services, die über das Netzwerk miteinander sprechen
- Mindestens ein zustandsbehafteter Dienst (Datenbank, Queue, Objektspeicher)
- Ein Lastpfad, den Sie künstlich erzeugen können
- Ein nachvollziehbarer fachlicher Zweck, der in einem Satz erklärbar ist

Ausgeschlossen sind reine CRUD-Demos ohne Lastpfad und Kopien von GridFlex mit umbenannten Feldern.

4. Pflichtthemen

Alle sieben müssen vorhanden und im Betrieb nachgewiesen sein. Sie decken den Kern der Vorlesung ab: ohne sie ist die Anwendung nicht cloud-native, sondern nur containerisiert.

ID	Thema	Nachweis
P1	Anwendung und Domäne	Zwei eigene Services und ein zustandsbehafteter Dienst, Container-Images bauen aus dem Repo
P2	Infrastruktur als Code	VMs, Netz und Cluster auf der DHBWCloud aus dem Repo erzeugt, kein Klick im Portal, ein Ablauf bringt alles hoch
P3	Deployment und Konfiguration	Requests und Limits, getaggte Images, Konfiguration über ConfigMap und Secret statt im Image
P4	Scheduling und Platzierung	Platzierung gesteuert und begründet, Verteilung der Pods über die Nodes wird gezeigt
P5	Resilienz und Persistenz	Probes, PVC oder StatefulSet, Rolling Update ohne Ausfall; Pod-Verlust wird überlebt, Daten bleiben
P6	Skalierbarkeit	HPA oder KEDA auf einer fachlich begründeten Metrik, Last erzeugt sichtbar Replicas
P7	Observability	<code>/metrics</code> -Endpunkt, ServiceMonitor, Grafana-Dashboard als Code, mindestens eine Alert-Regel, die auch auslöst

Zu P2: Provisionierung mit Terraform/OpenTofu gegen den OpenStack-Provider, Konfiguration mit Ansible. Andere Werkzeuge sind erlaubt, wenn Sie die Wahl begründen. Manuell im Portal angelegte Ressourcen, die im Code fehlen, zählen nicht.

Zu P4: mindestens eine harte Platzierungsentscheidung, die aus der Domäne folgt, nicht aus der Vorlesung. Requests und Limits sind so gewählt, dass Sie die Zahlen erklären können.

Zu P6: der Cluster hat mehrere Worker, damit Skalierung und Verteilung überhaupt sichtbar werden. Ein Ein-Knoten-Cluster erfüllt P4 und P6 nicht.

5. Wahlthemen

Genau zwei aus dieser Liste. Qualität über Quantität; ein drittes Thema bringt keine Punkte in Teil B, kann aber Bonuspunkte bringen (Abschnitt 8).

ID	Thema	Nachweis
W1	Packaging	Helm-Chart mit <code>values-dev.yaml</code> , <code>values-prod.yaml</code> und Values-Schema oder Kustomize mit Base und zwei Overlays
W2	GitOps	ArgoCD oder Flux, Auto-Sync aus Git, Drift wird zurückgedreht
W3	Progressive Delivery	Canary oder Blue-Green mit Umschaltpunkt und Rollback, Wirkung im Traffic messbar
W4	Traffic Management	API-Gateway mit Rate Limiting, Timeouts und Retries, Verhalten unter Überlast gezeigt
W5	Service Mesh	mTLS zwischen den Services, Golden Metrics aus dem Mesh, Retry- oder Timeout-Policy
W6	Authentifizierung	OIDC-Login und Token-Validierung am Gateway, nicht in der Anwendung; dazu RBAC im Cluster
W7	Cluster-Härtung	Network Policies nach Deny-by-default und Secret Management mit Sealed Secrets, SOPS oder Vault
W8	Supply-Chain-Security	Image-Scanning in der Pipeline, SBOM oder Signatur, Fund führt zu einer Konsequenz
W9	Container Patterns	Mindestens zwei von Sidecar, Ambassador, Adapter, Init-Container, fachlich nötig statt dekorativ
W10	Eigener Operator	Eigene CRD und Reconcile-Loop für eine nicht-triviale Aufgabe der Domäne, Drift wird zurückgeregelt
W11	Cluster Autoscaler	Node-Skalierung auf der DHBWCloud, Last erzeugt eine neue VM, Scale-down wird ebenfalls gezeigt

Zu W10: nicht-trivial heißt, der Operator tut etwas, das ein Deployment nicht kann. Ein Operator, der nur ein Deployment aus einer CRD erzeugt, zählt als Helm-Chart mit Umweg.

6. Abgabe

6.1 Git-Repository

Die Abgabe erfolgt pro Gruppe über ein Git-Repository. Es muss entweder als Link zu GitHub (öffentlich oder für den Github-User **pfisterer** lesbar) abgegeben werden oder als ZIP-Datei des Repositories (max. 100MB) in Moodle.

Das Git-Repository muss alle relevanten Artefakte (inkl. README.md, Screencast, Code, Konfigurationen, Diagramme) enthalten. Es muss so strukturiert sein, dass der Stack aus dem Repository heraus aufgebaut werden kann. Alle Konfigurationen, Skripte und Anweisungen müssen im Repository enthalten sein, um die Reproduzierbarkeit zu gewährleisten.

Der Commit-Verlauf muss nachvollziehbar sein, d.h. keine großen Monolith-Commits, sondern mehrere kleine Commits mit sinnvollen Nachrichten. Alle Nutzer der Gruppe sollen dort mit Ihren Beiträgen erkennbar sein. Es wird sich vorbehalten, auf Basis der Beiträge auch eine individuelle Bewertung vorzunehmen, wenn ein Gruppenmitglied deutlich weniger beigetragen hat.

6.2 README.md

Genau diese Abschnitte, in dieser Reihenfolge, als H2-Überschriften:

Abschnitt	Inhalt
## Problem	Die Domäne und warum sie ein Problem hat, in maximal 10 Zeilen
## Architektur	Diagramm und Datenfluss, von der VM bis zum Pod
## Betrieb	Wie man den Stack hochbringt, exakte Kommandos
## Entscheidungen	Je Pflicht- und Wahlthema ein Absatz: Wahl, Alternative, Begründung
## Grenzen	Was die Lösung nicht kann und was Sie unter echter Last erwarten
## Bonus	Nur falls Sie Bonuspunkte beanspruchen, sonst weglassen

6.3 Screencast

Maximal 7 Minuten, deutsch oder englisch, Bildschirmaufnahme mit Ton.

- Live-Terminal, keine vorproduzierten Standbilder
- Pflichtszene: Pod löschen, Anwendung erholt sich, Daten sind noch da
- Pflichtszene: Last erzeugen, Replicas wachsen, Wirkung im Grafana-Dashboard sichtbar

Nicht gefordert sind Folien, Intro-Animationen oder eine Vorstellungsrunde. Das Video soll so komprimiert werden, dass es in das 100MB-Limit des Repositories passt. Ein Link zu einem externen Video-Hoster ist nicht erlaubt. Bitte verwenden Sie Kompressionstools, wie z.B. HandBrake oder ffmpeg, um die Dateigröße zu reduzieren, ohne die Qualität zu stark zu beeinträchtigen.

7. Bewertungsschema

100 Punkte, bewertet aus Repository und Screencast.

Teil A: Pflichtthemen, 50 Punkte

Kriterium	Punkte
P1: zwei eigene Services und ein Stateful-Dienst, Images bauen reproduzierbar aus <code>src/</code>	6
P2: Infrastruktur und Cluster vollständig als Code, Stack kommt auf einem frischen Projekt hoch	10
P3: requests/limits, keine <code>:latest</code> -Tags, Konfiguration über ConfigMap und Secret ausgelagert	6
P4: Platzierung gesteuert und aus der Domäne begründet, Verteilung nachgewiesen	7
P5: Probes, PVC oder StatefulSet, Pod-Verlust und Update ohne Datenverlust überlebt	8
P6: Autoscaler auf begründeter Metrik, Skalierung unter Last demonstriert	7
P7: <code>/metrics</code> , ServiceMonitor, Dashboard als Code, Alert-Regel feuert	6

Wenn der Stack sich nicht aus dem Repository heraus aufbauen lässt, sind Teil A und Teil B nicht bewertbar. Probieren Sie das rechtzeitig auf einem frisch erzeugten Cluster, nicht auf Ihrem gewachsenen.

Teil B: Wahlthemen, 20 Punkte

Zwei Themen zu je 10 Punkten, jeweils:

Kriterium	Punkte
Ressourcen vorhanden und korrekt konfiguriert	4
Wirkung im Betrieb nachgewiesen	4
In <code>## Entscheidungen</code> fachlich begründet	2

Teil C: Dokumentation und Darstellung, 30 Punkte

Kriterium	Punkte
Alle geforderten Abschnitte vorhanden, Architekturdiagramm passt zum Deployment	4
Entscheidungen nennen je eine echte Alternative und einen Trade-off	7
Begründungen beziehen sich auf die Domäne, nicht auf die Vorlesung	4
Grenzen sind ehrlich und konkret benannt	2

Nachvollziehbarkeit: ein Fremder kommt der Reihe nach durch, ohne zu raten	4
Rechtschreibung und Grammatik	2
Qualität der Abbildungen	3
Screencast: Zeitlimit und Kapitelmarken stimmen, beide Pflichtszenen live, jedes Mitglied spricht	4

8. Bonuspunkte

Bis zu 10 Punkte für eine besondere Leistung, die über die Aufgabenstellung hinausgeht. Die Gesamtpunktzahl bleibt bei 100 gedeckelt; Bonuspunkte gleichen also Schwächen an anderer Stelle aus, sie erzeugen keine Note besser als 1,0.

Voraussetzung: Teil A ist vollständig erbracht. Ein Bonus für Fleiß auf Kosten der Pflichtthemen gibt es nicht.

ID	Leistung	Was zählt	Punkte
B1	Drittes Wahlthema	Vollständig wie in Teil B, inklusive Nachweis im Betrieb und Begründung	bis 5
B2	Neue Technologie	Eine cloud-native Technologie, die in der Vorlesung nicht behandelt wurde, im Stack tragend eingesetzt	bis 6
B3	AI-Workload	Function-calling-fähiges Small Language Model im Cluster über OpenAI/oder ähnlich-kompatible API, Ressourcenprofil je Umgebung	bis 6
B4	Belastbare Messung	Lastreihe mit Messpunkten, Grafik und Interpretation	bis 5
B5	Chaos-Experiment	Node oder Dienst gezielt zerstört, Erwartung vorher formuliert, Abweichung diskutiert	bis 5
B6	Wiederverwendbarer Baustein	Eigene Ansible-Rolle, eigenes Terraform-Modul oder eigenes Chart, veröffentlicht und von Dritten nutzbar dokumentiert	bis 5
B7	Eigener Vorschlag	Etwas, das Sie für bemerkenswert halten; vorher per Mail mit mir abgestimmt	bis 5

Zu B2, als Inspiration und nicht als Katalog :

Richtung	Beispiele
Tracing	OpenTelemetry mit Tempo oder Jaeger, eine Anfrage über beide Services verfolgt
Policy as Code	Kyverno oder OPA Gatekeeper, ein Verstoß wird beim Apply abgelehnt
eBPF-Netzwerk	Cilium mit Hubble, L7-Policies und sichtbarer Flow zwischen den Services
Chaos-Werkzeug	Chaos Mesh oder LitmusChaos als Experiment im Cluster statt <code>kubectll delete pod</code>
Backup und Restore	Velero, Namespace gelöscht und aus dem Backup zurückgeholt
Serverless	Knative oder OpenFaaS für einen Pfad, Scale-to-zero gezeigt
Infrastruktur-CRDs	Crossplane, OpenStack-Ressourcen aus dem Cluster heraus verwaltet
Continuous Profiling	Pyroscope, ein Hotspot im Code gefunden und behoben
Service Level Objective (SLOs)	Sloth oder Pyrra, Error Budget und Burn-Rate-Alert aus Ihren Metriken
Kosten	OpenCost, Kosten je Namespace gegen die Skalierungsentscheidung gestellt
WASM	SpinKube oder wasmCloud für einen Workload, Startzeit gegen den Container gemessen
Kafka-Betrieb	Strimzi über den Beispielumfang der Vorlesung hinaus

Beanspruchen Sie den Bonus im Abschnitt **## Bonus**: je Leistung ein Absatz mit ID, was Sie getan haben und wo es im Repository liegt. Nicht beanspruchte Leistungen suche ich nicht. Bonuspunkte sind eine Ermessensentscheidung; ein erfüllter Haken allein reicht nicht, es zählt die Tiefe. Eine Technologie zählt nur einmal, entweder als Wahlthema oder als Bonus.

9. Formales

- Fremder Code ist erlaubt, wenn er in **## Entscheidungen** als solcher benannt wird
- KI-Werkzeuge sind erlaubt. Sie müssen dennoch jede Zeile im Repository erklären können
- Ein Repository, das nur aus GridFlex mit ersetzten Bezeichnern besteht, wird mit 5,0 bewertet
- Verspätete Abgabe: 5,0, außer bei attestierten Gründen